Practicals for 5th Sem (1-10)

Practical 1: Classes and Objects

Create a Java class named **Student** with fields for roll number, name, and marks in three subjects. Add a constructor and methods to calculate total marks, percentage, and display the student details. Create at least three Student objects and display their details. Identify the class, objects, instance variables, constructor, and methods in your program.

Practical 2: Single, Multilevel, and Hierarchical Inheritance

Write Java programs to demonstrate all three inheritance structures. For single inheritance, create **Vehicle** and **Car** classes. For multilevel inheritance, create **Person**, **Employee**, and **Manager** classes. For hierarchical inheritance, create a **Shape** base class with **Circle** and **Rectangle** subclasses. Add suitable fields and methods, then demonstrate inherited behaviour and display the output.

Practical 3: Hybrid and Multiple Inheritance Using Interfaces

Create interfaces **Printable** and **Scannable**, each with an appropriate method. Implement both interfaces in an **AllInOnePrinter** class and demonstrate both behaviours. Then create a hybrid structure using a base class, a derived class, and one or more interfaces. Demonstrate the methods through objects and explain why Java supports multiple inheritance through interfaces but not through multiple parent classes.

Practical 4: Method Overloading and Method Overriding

Create a **Calculator** class with overloaded methods named **add()** that accept different numbers or types of arguments. Next, create a base class with a method named **calculatePay()** and derive two classes that override it with different calculations. Call the overloaded methods with different arguments and demonstrate overriding through a base-class reference. State which example demonstrates compile-time polymorphism and which demonstrates runtime polymorphism.

Practical 5: Simulating Operator Overloading Using Methods in Java

Create a **ComplexNumber** class containing real and imaginary parts. Implement an **add()** method that accepts another ComplexNumber and returns a new object containing their sum. Add a method to display a complex number in a readable format. Create two objects, add them, and display both inputs and the result. Explain why Java does not allow programmers to define custom operator overloading for user-defined classes.

Practicals for 5th Sem (1-10)

Practical 6: Default, Parameterized, and Copy Constructors

Create a **Book** class with fields for title, author, and price. Implement a no-argument constructor, a parameterized constructor, and a user-defined copy constructor that receives another Book object. Create objects using all three constructors and display their values. Modify one object after copying it and observe whether the original object changes. Briefly explain how the three constructors differ.

Practical 7: Object Lifecycle, Garbage Collection, and Destructor Reference Exercise

Create a class that prints a message when an object is constructed. Create an object, use it, and then remove its reachable references. Request garbage collection and record what happens, without assuming that collection will occur immediately. Explain object eligibility for garbage collection, the role of Java memory management, and how a destructor in languages such as C++ differs from Java object cleanup. Do not rely on cleanup timing for program correctness.

Practical 8: Constructor Overloading

Create a **Product** class with fields for product name, price, and quantity. Provide overloaded constructors for: no arguments, product name only, product name and price, and all three fields. Use each constructor to create a different Product object. Add a method to calculate the total cost (price multiplied by quantity) and a method to display the object details. Show the output for every constructor variation.

Practical 9: this Keyword, Constructor Chaining, and Static Members

Create an **Employee** class with fields for employee name and salary, along with a static field that counts the number of employees created. Use **this.field** to distinguish instance fields from parameters. Make the no-argument constructor call a parameterized constructor using **this()**. Increment the static count whenever an object is constructed and provide a static method to display the count. Create multiple objects and demonstrate each feature.

Practical 10: Encapsulation Using Getters, Setters, and Validation

Create a **BankAccount** class with private fields for account holder, account number, and balance. Provide suitable getters and controlled methods for depositing and withdrawing money. Reject negative deposits, zero or negative withdrawals, and withdrawals greater than the available balance. In the main method, test valid and invalid transactions and display the updated balance. Demonstrate how private fields and validated methods protect an object from invalid state changes.